

Processamento de Lista Encadeada com Tasks em OpenMP

Israel Soares de Castro Filho
Matrícula: 20250070780

1 Introdução

Este relatório descreve a implementação e a análise de um programa em linguagem C que combina uma estrutura de dados clássica – a lista encadeada – com o modelo de paralelismo baseado em tarefas (*tasks*) da biblioteca OpenMP.

O objetivo é construir uma lista encadeada em que cada nó armazena o nome de um arquivo fictício e, dentro de uma região paralela, percorrer essa lista criando uma tarefa (`#pragma omp task`) responsável por processar cada nó individualmente.

Cada tarefa imprime o nome do arquivo que processou e o identificador da thread que a executou, obtido por meio da função `omp_get_thread_num()`. A partir dos resultados obtidos, este trabalho também discute questões relacionadas à correção do processamento paralelo, como a garantia de que cada nó seja processado exatamente uma vez e por uma única tarefa.

2 Metodologia

A lista encadeada é construída sequencialmente, antes da entrada na região paralela, com dez nós, cada um armazenando o nome de um arquivo fictício no formato `arquivo_NN.dat`.

Dentro da região `#pragma omp parallel`, o percurso da lista é feito por uma única thread, delimitada pela diretiva `#pragma omp single nowait`.

Essa escolha é fundamental: caso todas as threads do time percorressem a lista, cada uma criaria uma tarefa para cada nó, resultando em processamento duplicado (o nó seria processado tantas vezes quanto o número de threads). A cláusula `nowait` evita uma barreira desnecessária ao final do `single`, permitindo que as threads ociosas comecem a executar as tarefas já criadas assim que possível.

Para cada nó visitado, uma tarefa é criada com:

- `#pragma omp task firstprivate(no_da_vez)`.

O uso da cláusula `firstprivate` é essencial para a correção do programa: ela copia o valor do ponteiro para o nó no exato momento da criação da tarefa.

Sem essa cláusula, a variável de percurso seria compartilhada por padrão dentro da tarefa e poderia ser alterada pelo laço antes que a tarefa fosse efetivamente executada, fazendo com que múltiplas tarefas apontassem para o mesmo nó (tipicamente o último da lista).

Cada tarefa executa a função `processar_arquivo`, que imprime o nome do arquivo e o identificador da thread corrente (`omp_get_thread_num()`). Ao final da região paralela,

a barreira implícita garante que todas as tarefas tenham concluído sua execução antes que a memória da lista seja liberada. O código-fonte completo encontra-se no Apêndice A.

3 Resultados

A execução do programa produziu a seguinte saída, na ordem em que as linhas foram impressas:

```
Arquivo 'arquivo_02.dat' processado pela thread 3
Arquivo 'arquivo_05.dat' processado pela thread 1
Arquivo 'arquivo_04.dat' processado pela thread 4
Arquivo 'arquivo_06.dat' processado pela thread 3
Arquivo 'arquivo_03.dat' processado pela thread 5
Arquivo 'arquivo_01.dat' processado pela thread 2
Arquivo 'arquivo_07.dat' processado pela thread 1
Arquivo 'arquivo_08.dat' processado pela thread 2
Arquivo 'arquivo_09.dat' processado pela thread 6
Arquivo 'arquivo_10.dat' processado pela thread 9
```

A Tabela 1 reorganiza essa saída por ordem de arquivo, facilitando a verificação de que todos os nós foram processados.

Tabela 1: Arquivo processado e thread responsável

Arquivo	Thread
arquivo_01.dat	2
arquivo_02.dat	3
arquivo_03.dat	5
arquivo_04.dat	4
arquivo_05.dat	1
arquivo_06.dat	3
arquivo_07.dat	1
arquivo_08.dat	2
arquivo_09.dat	6
arquivo_10.dat	9

Verifica-se que os dez arquivos (`arquivo_01.dat` a `arquivo_10.dat`) aparecem exatamente uma vez cada na saída, totalizando as dez linhas esperadas. Nenhum arquivo foi omitido e nenhum foi processado mais de uma vez.

4 Análise

Cobertura e unicidade do processamento. Como mostrado na Tabela 1, cada um dos dez nós da lista foi processado exatamente uma vez, o que confirma a correção da estratégia adotada: a combinação de `single` (para que apenas uma thread crie as tarefas) com `firstprivate` (para que cada tarefa capture o ponteiro correto do nó) evita tanto a duplicação quanto a omissão de nós.

Distribuição entre threads. A distribuição das tarefas entre as threads não é uniforme: as threads 1, 2 e 3 processaram dois arquivos cada, enquanto as threads 4, 5, 6 e 9 processaram apenas um arquivo cada. Essa distribuição desigual é esperada,

pois o escalonador de tarefas do OpenMP atribui as tarefas dinamicamente às threads ociosas conforme elas terminam seu trabalho anterior – não há garantia de balanceamento perfeito, especialmente com uma carga de trabalho tão pequena (dez tarefas curtas). A presença de identificadores de thread não sequenciais entre os que aparecem na saída (por exemplo, a ausência das threads 0, 7 e 8 no conjunto observado) apenas reflete que, para essa execução específica, nem todas as threads do time chegaram a receber uma tarefa antes que a lista fosse totalmente percorrida.

Ordem de impressão. A ordem em que as linhas aparecem na saída (por exemplo, `arquivo_02.dat` antes de `arquivo_01.dat`) não corresponde à ordem dos nós na lista encadeada. Isso é uma consequência natural do paralelismo: as tarefas são criadas na ordem da lista, mas sua execução e a impressão do resultado dependem de quando cada thread fica disponível para executá-las. Em execuções repetidas do mesmo programa, é esperado que essa ordem varie, assim como pode variar qual thread processa qual arquivo, embora o conjunto de arquivos processados (exatamente uma vez cada) permaneça sempre o mesmo, por construção.

Garantia de processamento único. A robustez do programa depende de três elementos estruturais, e não de sorte na execução: (i) o percurso da lista restrito a uma única thread via `single`, evitando que múltiplas threads criem tarefas para o mesmo conjunto de nós; (ii) a cláusula `firstprivate` na variável que referencia o nó atual, garantindo que cada tarefa opere sobre o nó correto mesmo que a variável de laço mude antes da execução efetiva da tarefa; e (iii) a barreira implícita ao final da região `parallel` (que poderia também ser obtida explicitamente com `#pragma omp taskwait`), assegurando que todas as tarefas concluam antes que recursos como a lista sejam liberados.

5 Conclusão

O experimento confirmou que é possível processar de forma paralela e correta cada nó de uma lista encadeada utilizando tarefas OpenMP, desde que sejam respeitadas as práticas de criação de tarefas em uma região paralela: percurso serial da lista dentro de um bloco `single` e captura por valor (`firstprivate`) da referência ao nó em cada tarefa criada. Nos resultados obtidos, todos os dez arquivos foram processados exatamente uma vez, sem duplicações ou omissões, ainda que a ordem de processamento e a thread responsável por cada arquivo variem entre execuções devido à natureza dinâmica do escalonador de tarefas do OpenMP. Esse comportamento é esperado e não compromete a corretude do programa, pois a garantia de unicidade decorre da estrutura do código e não da ordem de execução das threads.

A Código-Fonte

O código-fonte completo utilizado nesta atividade está disponível em `../src/tarefa7.c` e é reproduzido abaixo.

```
1  /*
2   * Exemplo: lista encadeada + OpenMP tasks
3   *
4   * Compilar: gcc -fopenmp -Wall -o lista_tasks lista_tasks.c
5   * Executar: ./lista_tasks
6   */
7
8  #include <stdio.h>
9  #include <stdlib.h>
10 #include <string.h>
11 #include <omp.h>
12
13 typedef struct Node {
14     char nome_arquivo[64];
15     struct Node *prox;
16 } Node;
17
18 /* Cria um novo nó com o nome do arquivo informado */
19 Node *criar_no(const char *nome) {
20     Node *n = (Node *) malloc(sizeof(Node));
21     if (!n) {
22         fprintf(stderr, "Erro de alocação de memória\n");
23         exit(EXIT_FAILURE);
24     }
25     strncpy(n->nome_arquivo, nome, sizeof(n->nome_arquivo) - 1);
26     n->nome_arquivo[sizeof(n->nome_arquivo) - 1] = '\0';
27     n->prox = NULL;
28     return n;
29 }
30
31 /* Insere um novo nó no final da lista */
32 void inserir(Node **cabeca, const char *nome) {
33     Node *novo = criar_no(nome);
34     if (*cabeca == NULL) {
35         *cabeca = novo;
36         return;
37     }
38     Node *atual = *cabeca;
39     while (atual->prox != NULL) {
40         atual = atual->prox;
41     }
42     atual->prox = novo;
43 }
44
45 /* Libera toda a memória da lista */
46 void liberar_lista(Node *cabeca) {
47     Node *tmp;
48     while (cabeca != NULL) {
49         tmp = cabeca;
50         cabeca = cabeca->prox;
51         free(tmp);
52     }
53 }
```

```

54
55 /* "Processa" um nó */
56 void processar_arquivo(const char *nome_arquivo) {
57     printf("Arquivo '%s' processado pela thread %d\n",
58           nome_arquivo, omp_get_thread_num());
59 }
60
61 int main(void) {
62     Node *lista = NULL;
63     char nome[32];
64     int total_nos = 10;
65
66     /* Monta a lista encadeada */
67     for (int i = 1; i <= total_nos; i++) {
68         snprintf(nome, sizeof(nome), "arquivo_%02d.dat", i);
69         inserir(&lista, nome);
70     }
71
72     #pragma omp parallel
73     {
74         /* IMPORTANTE:
75         * "single" garante que APENAS UMA thread percorre a lista e
76         * cria as tasks. Sem isso, todas as threads da região paralela
77         * executariam o laço, cada uma criando uma task para CADA nó,
78         * resultando em processamento duplicado (N vezes, sendo N o
79         * número de threads).
80         *
81         * "nowait" evita uma barreira implícita ao final do single,
82         * permitindo que as threads ociosas comecem a executar as
83         * tasks já criadas assim que possível.
84         */
85         #pragma omp single nowait
86         {
87             Node *atual = lista;
88             while (atual != NULL) {
89                 Node *node_atual = atual; /* cópia local para a task */
90
91                 #pragma omp task firstprivate(node_atual)
92                 {
93                     processar_arquivo(node_atual->nome_arquivo);
94                 }
95
96                 atual = atual->prox;
97             }
98         }
99     }
100
101     liberar_lista(lista);
102     return 0;
103 }

```

Código 1: Código-fonte completo (tarefa7.c)